Memory Hierarchy

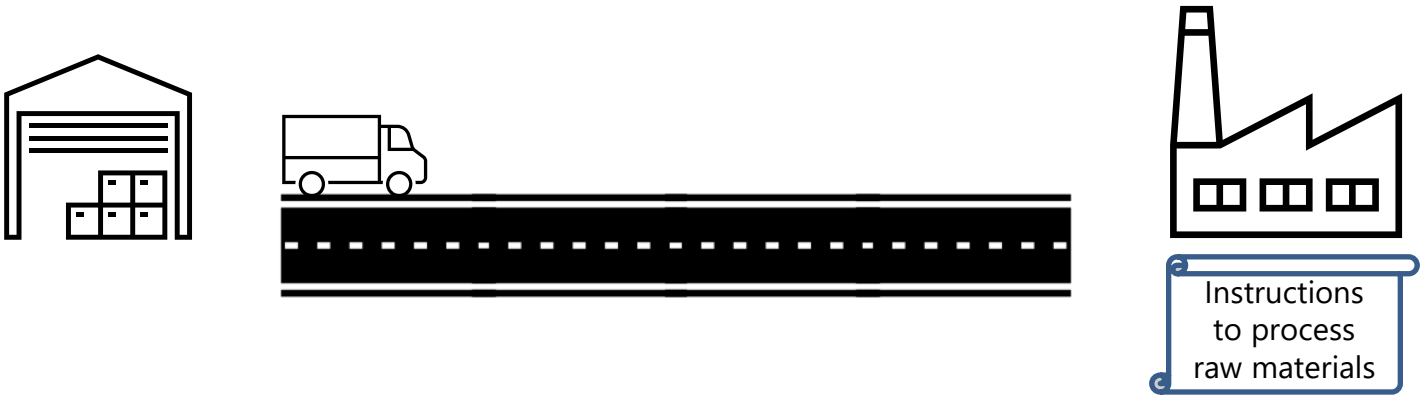
CS 2410

Wonsun Ahn

University of Pittsburgh

Impact of Memory On Performance

Remember our factory analogy? Memory is Logistics!



- If you can't move goods in a timely manner your output will suffer
- Factory efficiency doesn't matter if logistics is the bottleneck
  - If deliveries are slow (high memory latency)
  - Or do not arrive in sufficient quantities (low memory bandwidth)

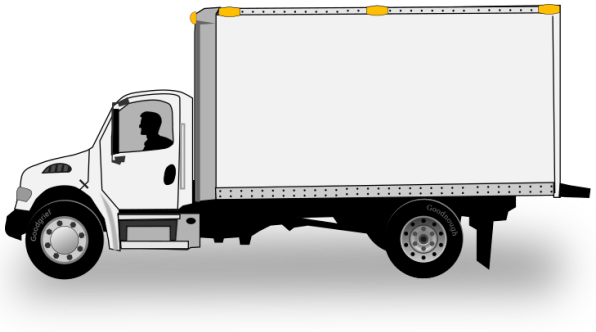
Memory Latency vs. Memory Bandwidth

- Memory bottleneck comes from two sources:
  - **Memory latency**: hours to handle a single delivery

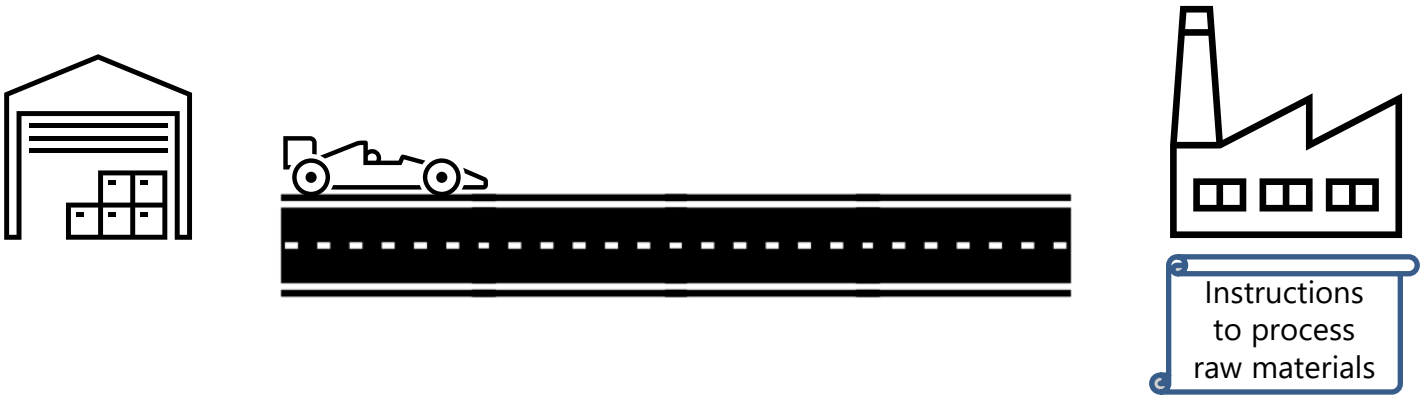
vs.

  - **Memory bandwidth**: maximum volume handled per hour

vs.



Memory Latency does not cap performance

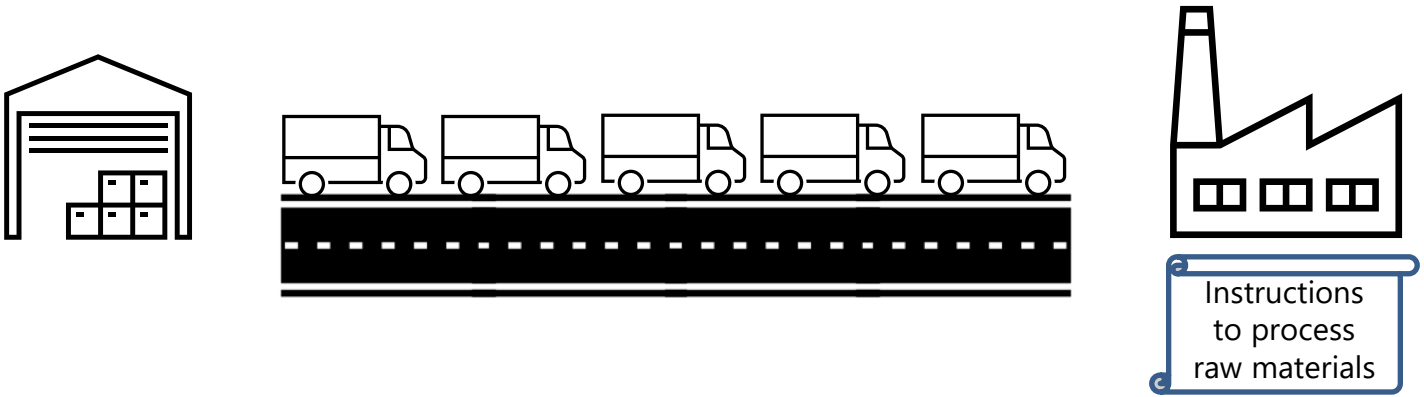


- Race cars are rarely used for logistics. Trucks are. Why?

1. The factory can produce something else while waiting (*scheduling*)
2. Can predict future orders based on past orders (*prefetching*)

→ Memory Latency does not put a hard limit on performance.

But Memory Bandwidth does cap performance



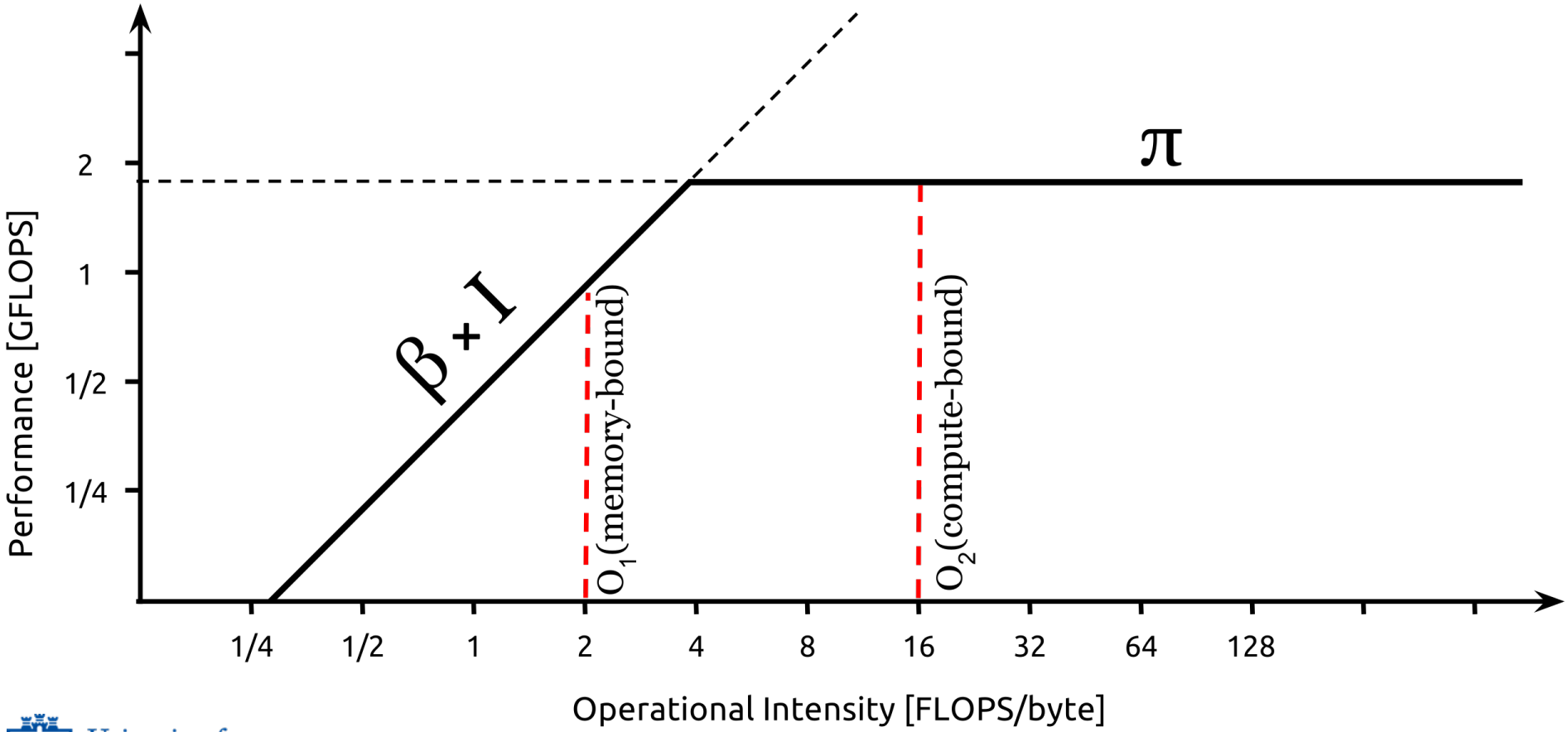
- If the link can only handle only 1 truck / minute, that dictates throughput.

1. Per minute, factory can only do 1 truck load worth of work
2. Only way to increase productivity is to increase **operational intensity**.

→ Memory Bandwidth does put a hard limit on performance.

Memory Bandwidth forms the diagonal of the Roofline

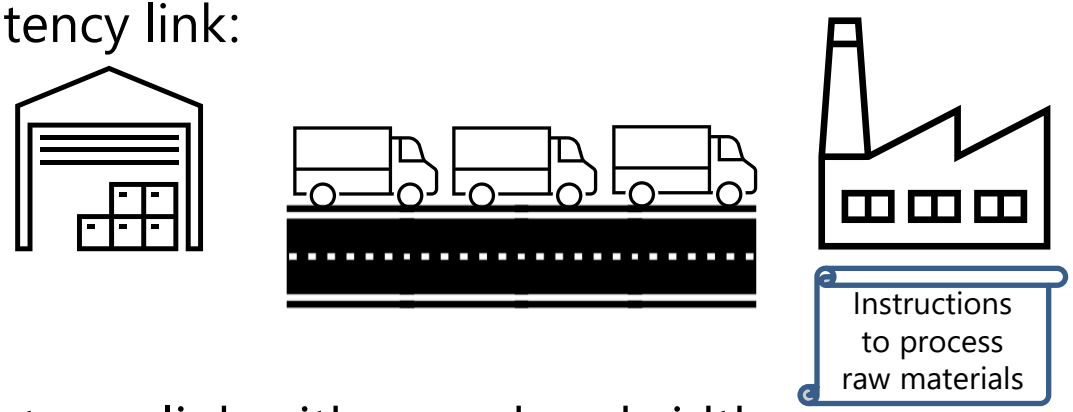
- I = intensity, β = peak bandwidth, π = peak performance



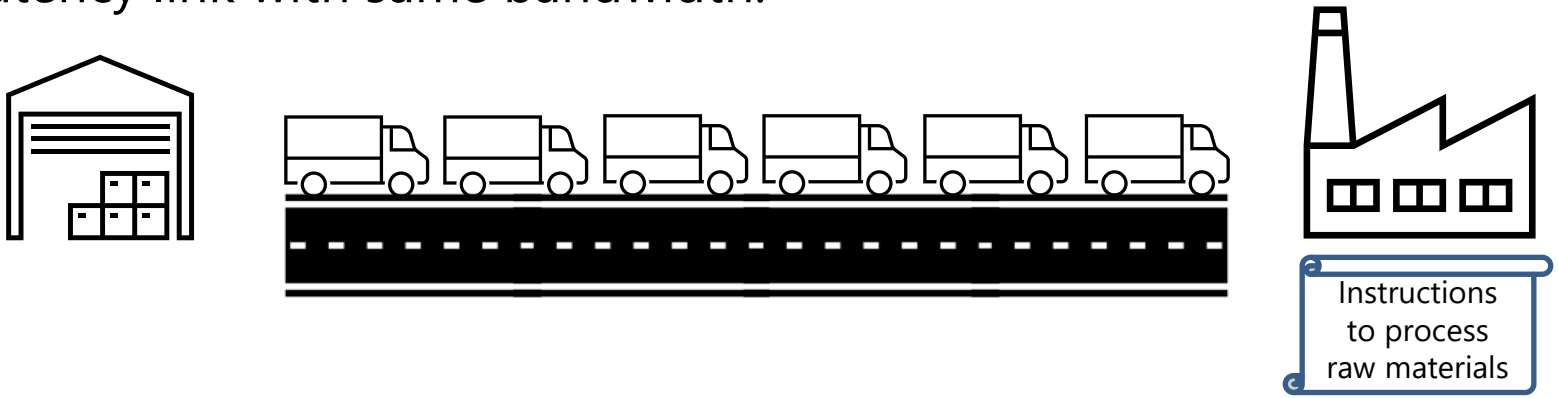
University of Pittsburgh

Note: Memory Bandwidth ≠ Memory Latency

- Low latency link:



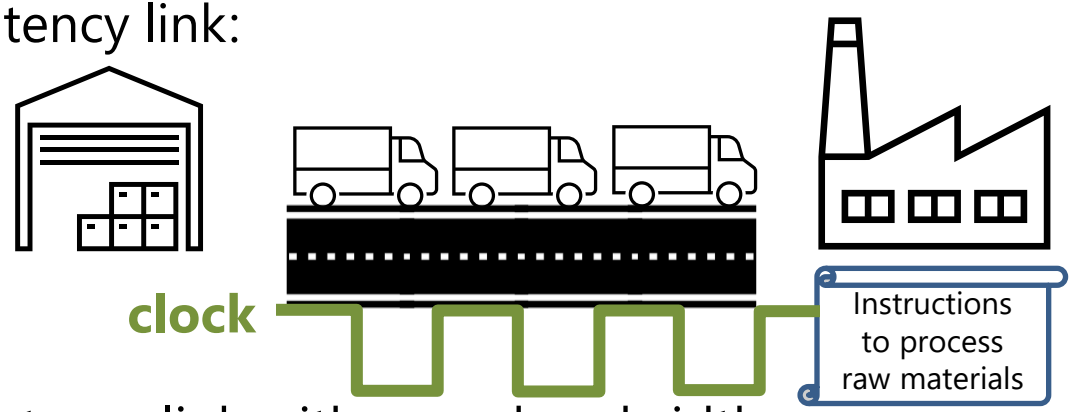
- High latency link with same bandwidth:



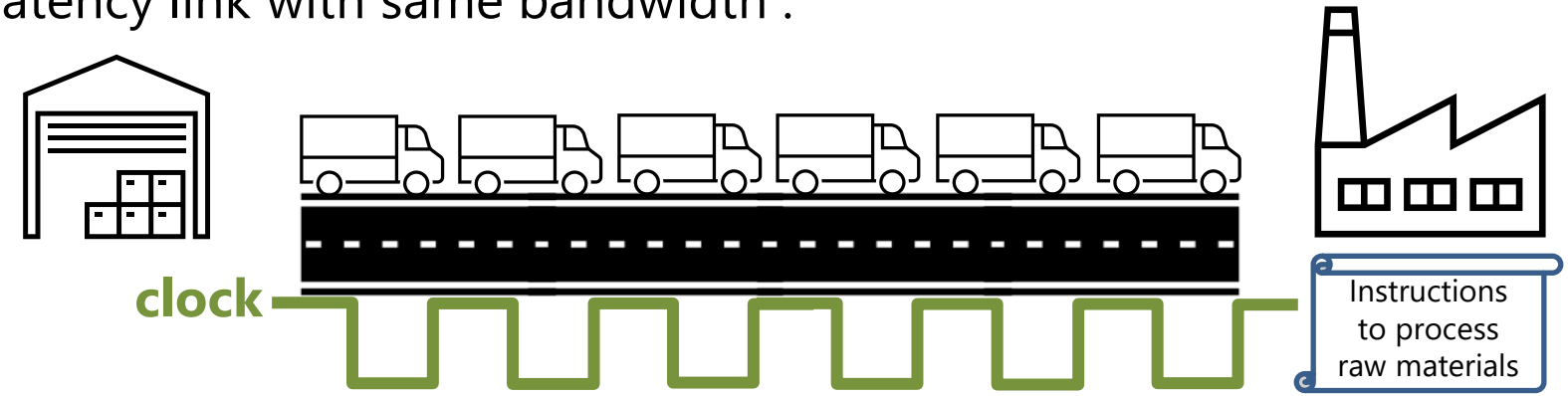
- If both links handle 1 truck / minute, both have the **same bandwidth**.

Bandwidth ≠ Latency, that is given a pipelined link

- Low latency link:



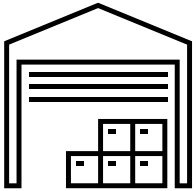
- High latency link with same bandwidth :

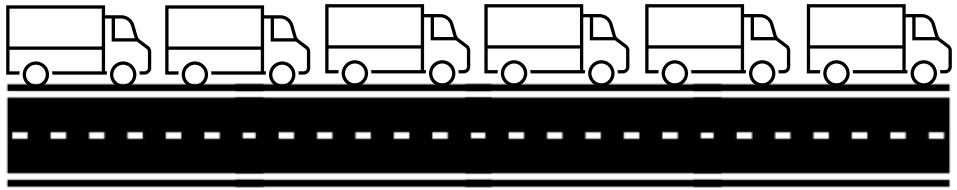


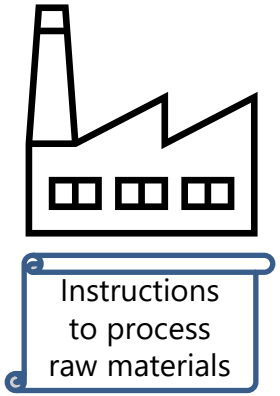
- Possible because of **pipelined** link (multiple data packets can be in flight).

Keeping Link Pipeline filled is Contingent Upon MLP

- Can you always keep the road filled with trucks?



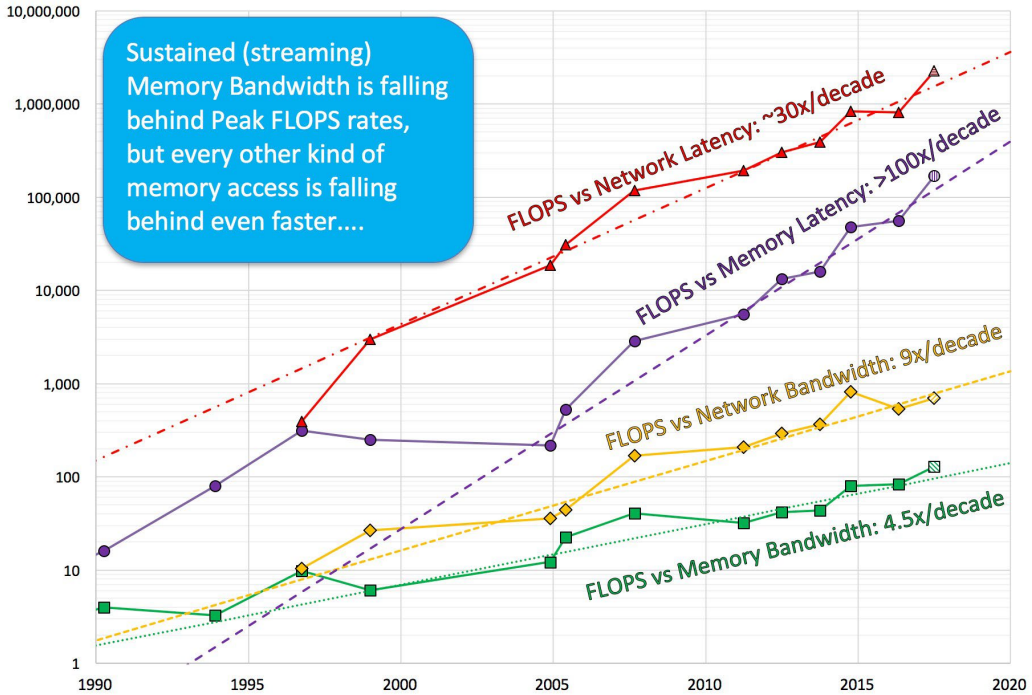




- Highly dependent upon whether there are dependencies between trucks!

- **Memory Level Parallelism** (MLP): degree of concurrency in data accesses
  - Just like ILP utilizes wide CPU, MLP utilizes high-throughput link.
  - Just like ILP, MLP is an inherent property of the program.
    (E.g. linked-list traversal → low MLP vs. array traversal → high MLP)
  - Just like ILP, CPU needs a large instruction window to extract MLP.

Memory Wall: widening gap between memory and CPU

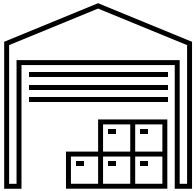


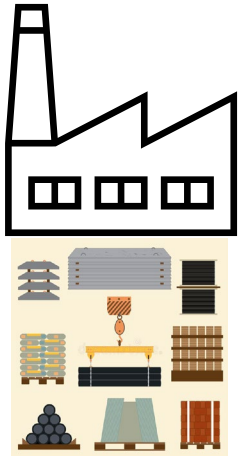
Source: SC16 Invited Talk ""Memory Bandwidth and System Balance in HPC Systems" by John D. McCalpin

- FLOPS = floating point operations per second (CPU speed)

University of Pittsburgh

Cache: storage in CPU for frequently accessed data

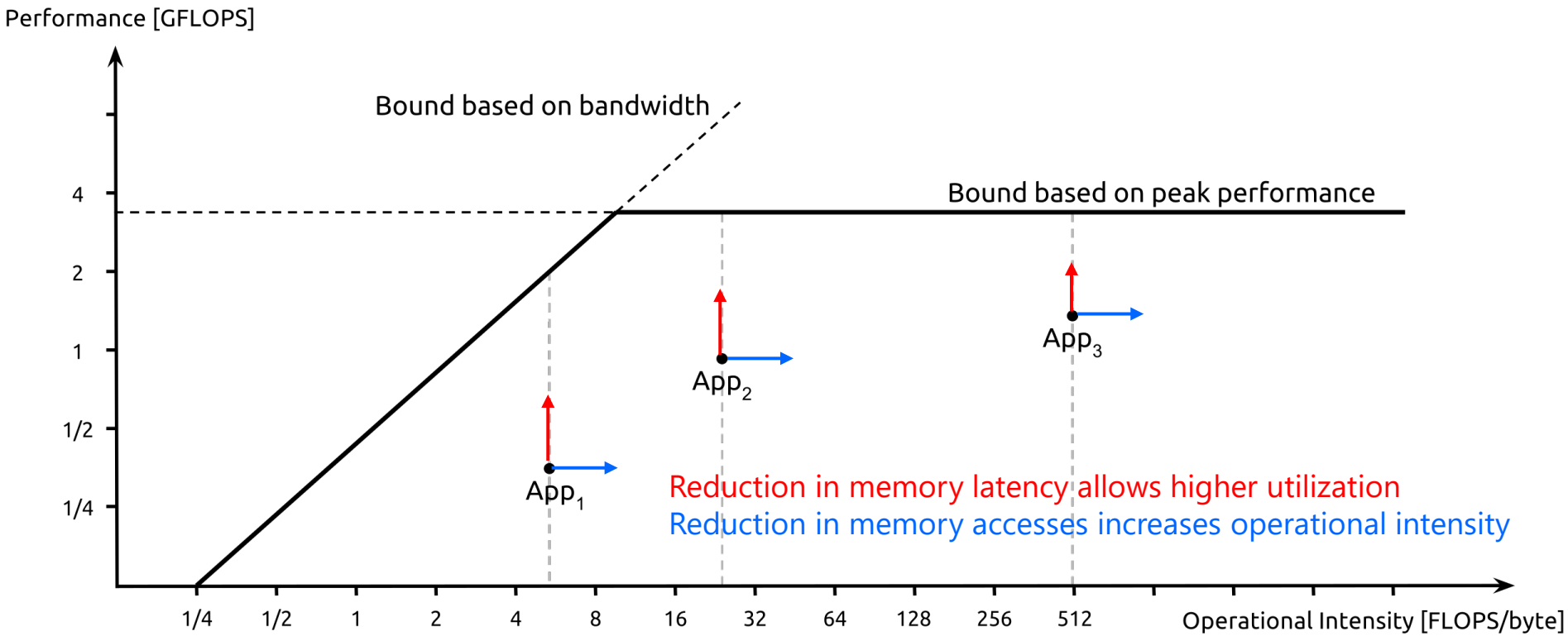




- What if we had a storage **cache** in factory premises?
  - Store frequently used inventory nearby
- Then, both memory latency and bandwidth problems are alleviated
  - Latency: access is quick when material is found in the cache
  - Bandwidth: less need to go back and forth to the warehouse

Effect of Cache on Roofline Model

- With caching, apps shift upwards and rightwards



CPU Cycles = CPU Compute Cycles + Memory Stall Cycles

- **CPU Cycles = CPU Compute Cycles + Memory Stall Cycles**
  - CPU Compute Cycles = cycles where CPU is not stalled on memory
  - Memory Stall Cycles = cycles where CPU is waiting for memory

- HW/SW ways to reduce **CPU Compute Cycles**:
  - HW: Pipelining, branch prediction, wide execution, out-of-order
  - SW: Redundant instruction elimination, scheduling, increasing ILP

- HW/SW ways to reduce **Memory Stall Cycles**:
  - HW: Caches, write buffer, prefetcher (we haven't learned these yet)
  - SW: Optimize memory accesses for better MLP or cache use

Impact of overclocking (DVFS) on Memory Stall Time

- CPU Time = CPU Cycles * Cycle Time
  = CPU Compute Cycles * Cycle Time + Memory Stall Cycles * Cycle Time

- What if we halved the Cycle Time using DVFS?

- Impact on CPU Compute Cycles * Cycle Time:
  - CPU Compute Cycles remains constant (no change in bubbles)
  - CPU Compute Time is also reduced to half!

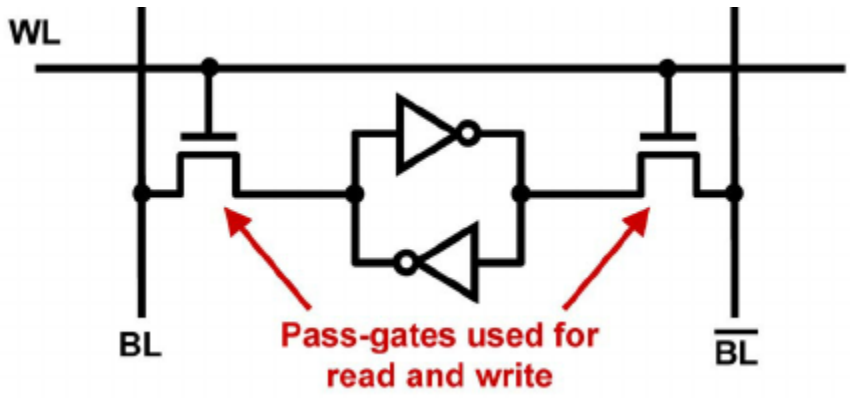
- Impact on Memory Stall Cycles * Cycle Time:
  - Memory Stall Cycles increases by 2X (if no change in cache misses)
  - Why? DRAM speed remains same → Need twice the cycles per access
    - The bus (wire) that connects CPU to DRAM is not getting any faster
    - The DRAM chip itself is not getting any faster
  - End result may be there is no improvement in performance

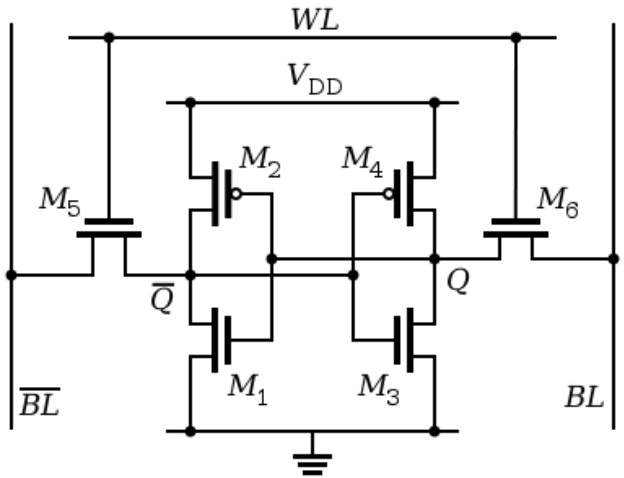
Memory Technologies

Static RAM (SRAM)

- SRAM uses a loop of NOT gates
  - In order to store a single bit
- This is called a 6T SRAM cell
  - Because it uses… **6 transistors**!

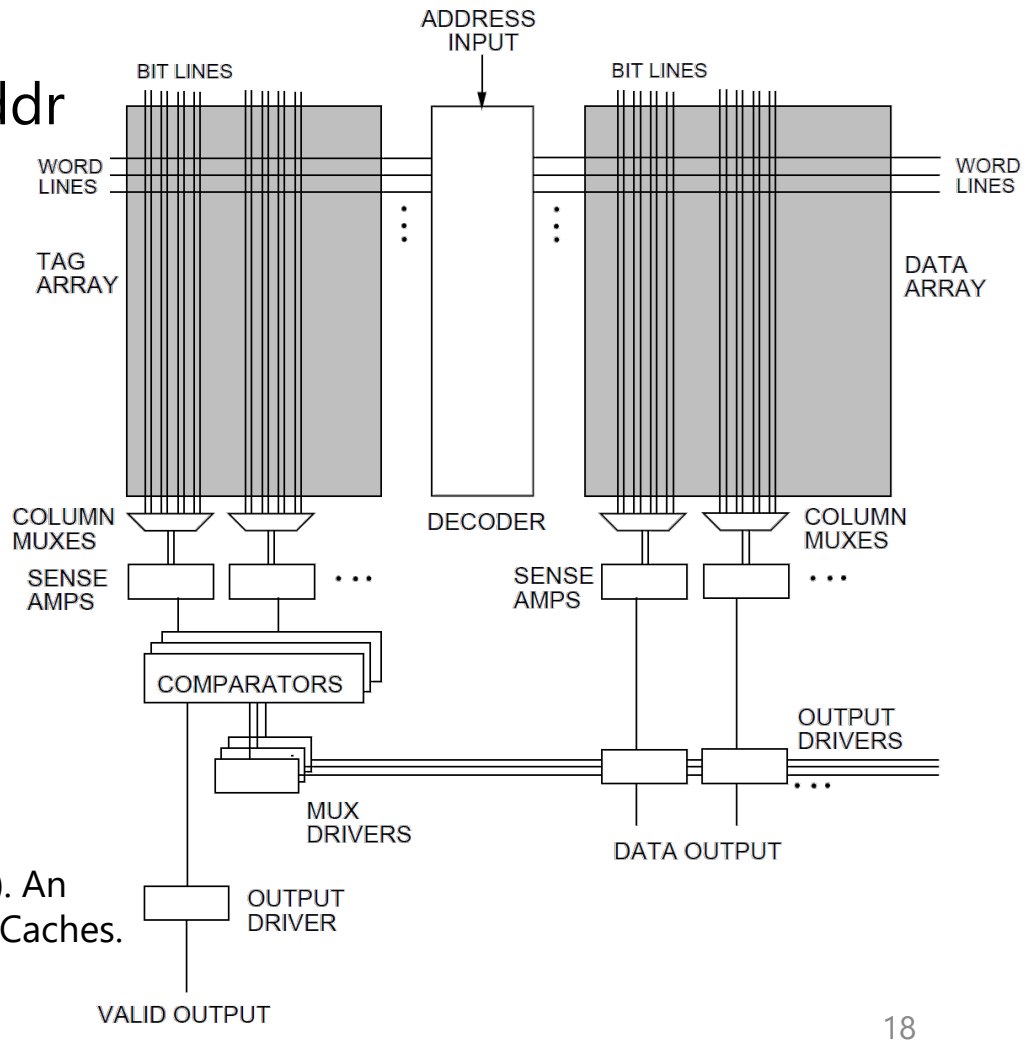
- **Pros:**
  - **Very fast** to read/write
- **Cons:**
  - Volatile (loses data without power)
  - Relatively many transistors needed -> **expensive**
- **Uses:**
  - CPU registers
  - CPU caches





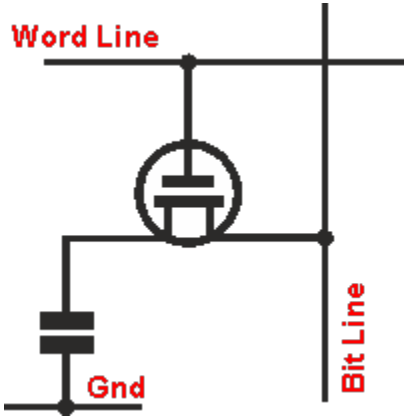
SRAM Cache Structure

- Memory is a 2D array
  - Word lines select row from addr
  - Bit lines read out bits of word
- Each row contains
  - Tag bits (when needed)
    - Left shaded square
    - Resolves hash conflicts
  - Data bits
    - Right shaded square



Courtesy: Winton, Steven and Jouppi, Norman. (1994). An Enhanced Access and Cycle Time Model for On-Chip Caches.

Dynamic RAM (DRAM)



- DRAM uses **one** transistor and **one** capacitor
  - The bit is stored as a charge in the capacitor
  - Capacitor leaks charge over time
    -> Must be periodically recharged (called **refresh**)
    -> During refresh, DRAM can't be accessed
  - Accesses are slower
    -> Small charge must be amplified to be read
    -> Also after read, capacitor needs recharging again
  - Reading a DRAM cell is slower than reading SRAM
- **Pros:**
  - Higher density -> less silicon -> **much cheaper than SRAM**
- **Cons:**
  - Still volatile (even more volatile than SRAM)
  - **Slower access time**

Other technology

- NAND Flash Memory
  - Works using a special MOSFET with “floating gate”
  - **Pros**: nonvolatile, much faster than HDD
  - **Cons**:
    - Slower than DRAM
    - Accesses at page (~4K) granularity → replaces HDD nor DRAM
    - More expensive than HDDs (1TB for ~$150)
    - Writing is destructive and shortens lifespan

- Experimental technology in varying states of maturity
  - Ferroelectric RAM (FeRAM), Magnetoresistive RAM (MRAM), Phase-change RAM (PRAM), Resistive RAM (ReRAM) …
  - Nonvolatile *and* allows byte granularity access just like DRAM
  - PRAM and ReRAM can store a continuous resistance, not just binary
    - Can do in-memory neural computing using resistances as weights (Using Ohm’s law: I = V / R). See IBM Hermes chip.

Spinning magnetic disks (HDD)

- Spinning platter coated with a ferromagnetic substance magnetized to represent bits
  - Has a mechanical arm with a head
  - Reads by placing arm in correct cylinder, and waiting for platter to rotate

- **Pros:**
  - **Nonvolatile** (magnetization persists without power)
  - Extremely cheap (1TB for ~$50)
- **Cons:**
  - **Extremely slow** (it has a mechanical arm, enough said)

1. There is a trade-off between speed and cost

	Volatile		Nonvolatile	
	SRAM	DRAM	Flash	HDD
Speed	**FAST**	Pretty fast	OK	**SLOW**
Price	**Expensive**	OK	Pretty cheap	**Cheap**

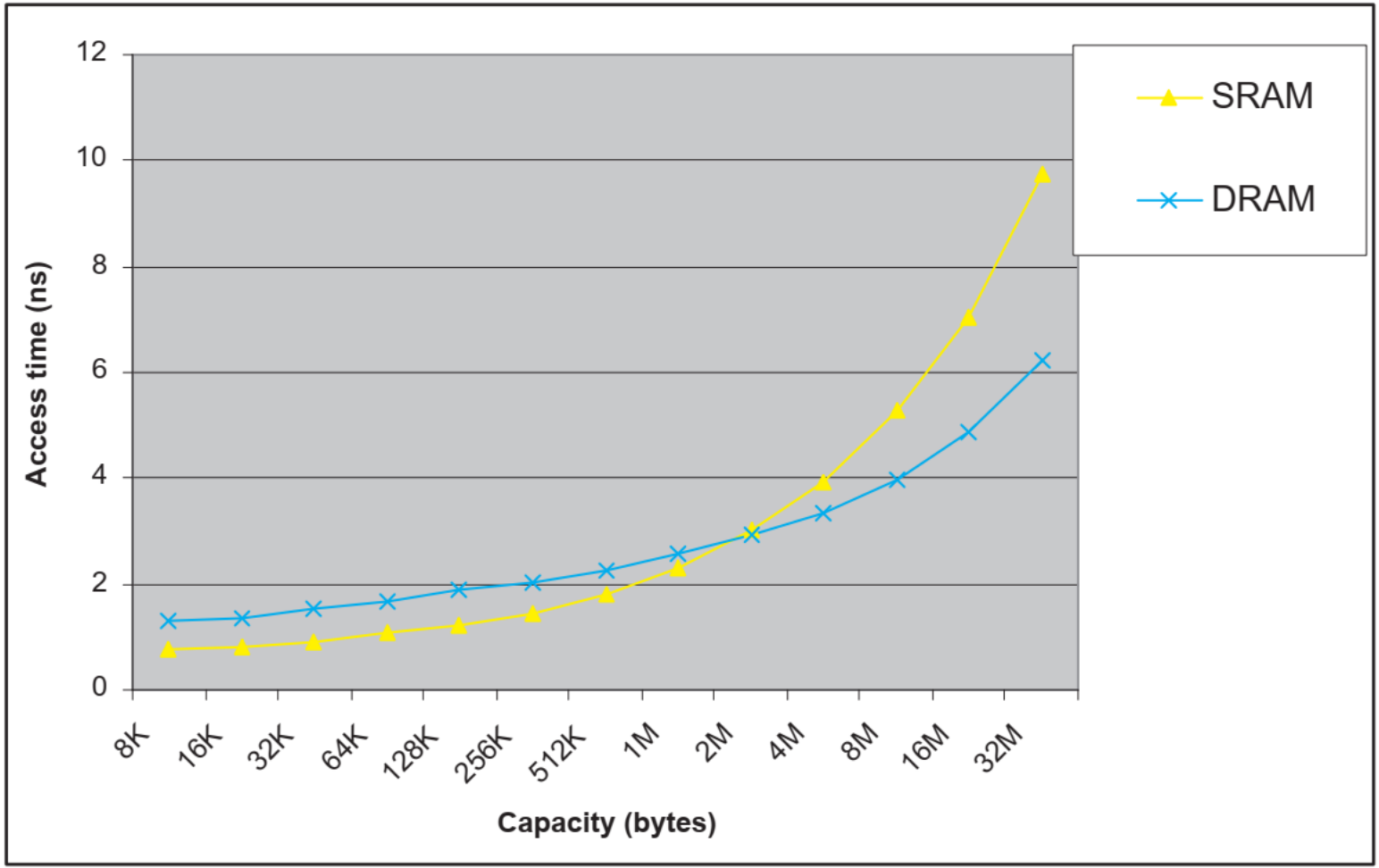
SLOWER! →

CHEAPER! →

- Slower memory tends to be cheaper.
- Faster memory tends to be more expensive.

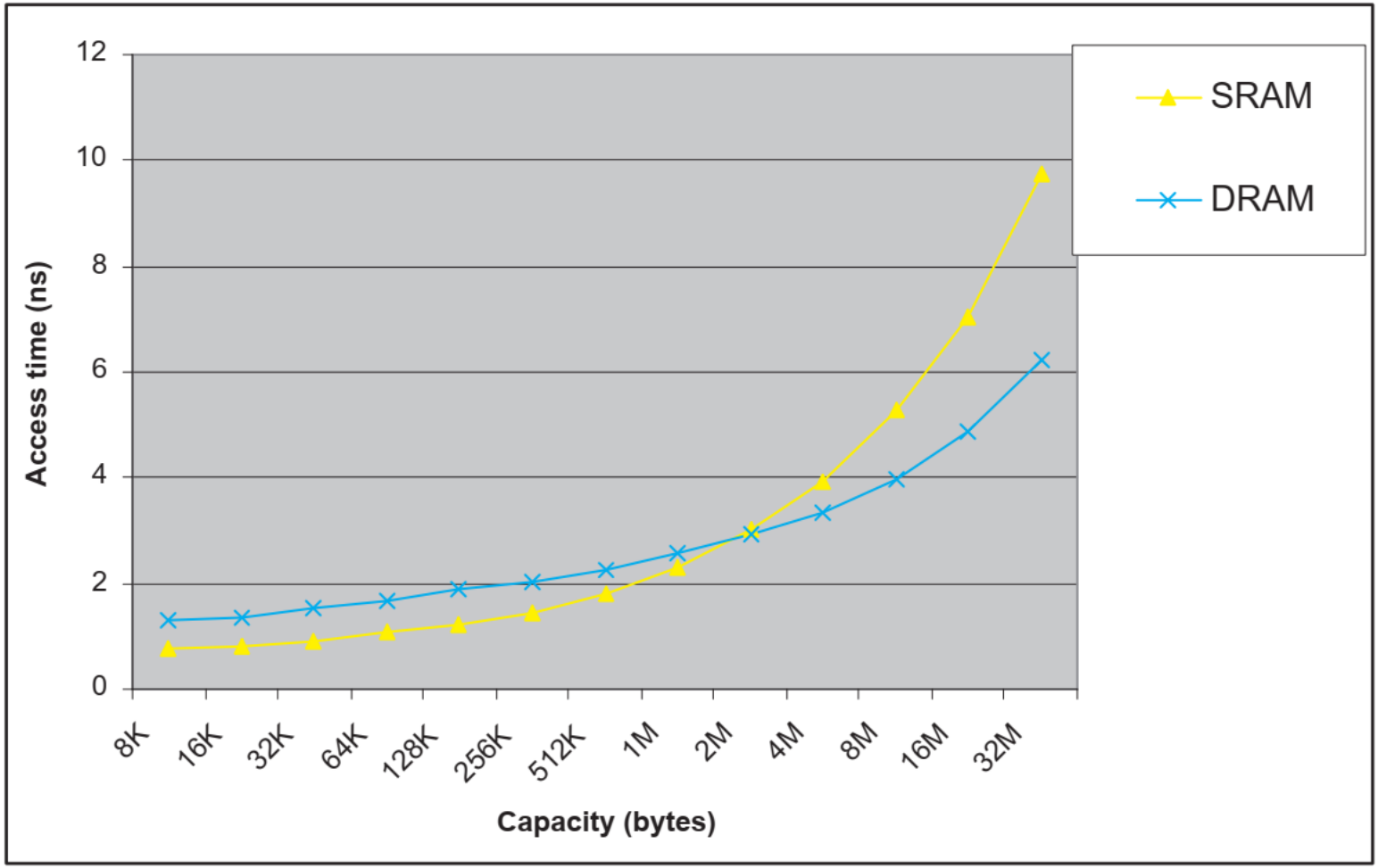
- What kind of memory should we use for **caches**?

2. With same memory technology, larger is slower



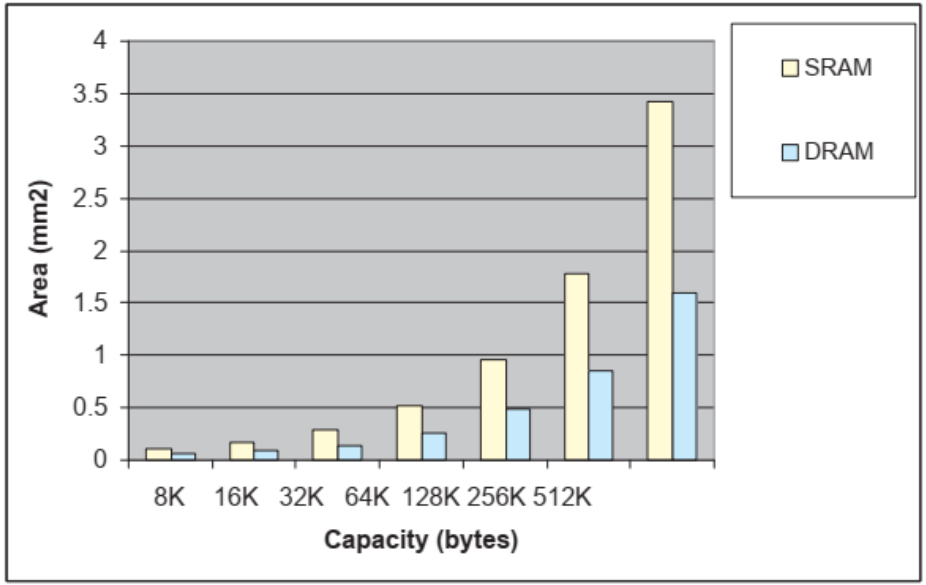
Thoziyoor, Shyamkumar & Muralimanohar, Naveen & Ahn, Jung Ho & Jouppi, Norman. (2008). CACTI 5.1.

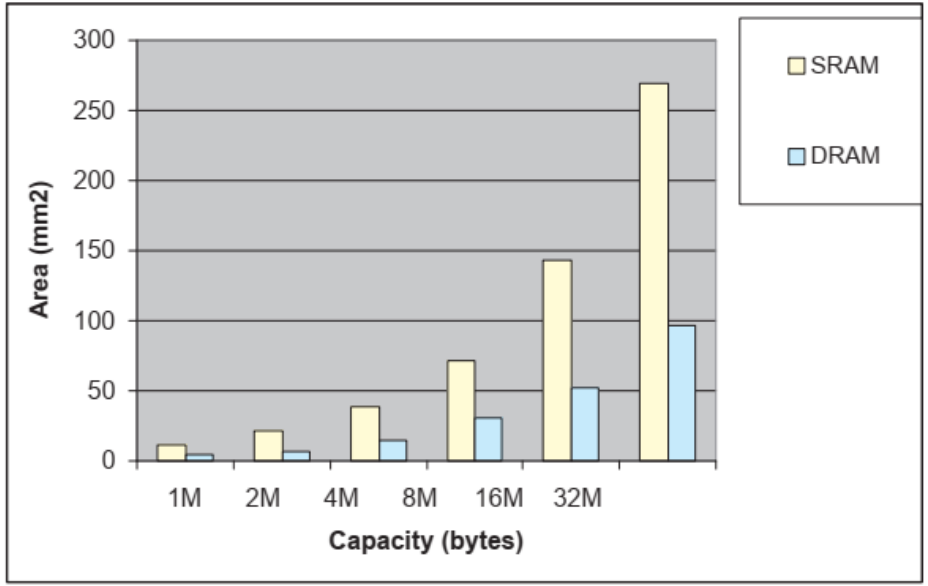
3. DRAM is faster than SRAM at high capacity



Thoziyoor, Shyamkumar & Muralimanohar, Naveen & Ahn, Jung Ho & Jouppi, Norman. (2008). CACTI 5.1.

Due to wire delay dominating at high capacities



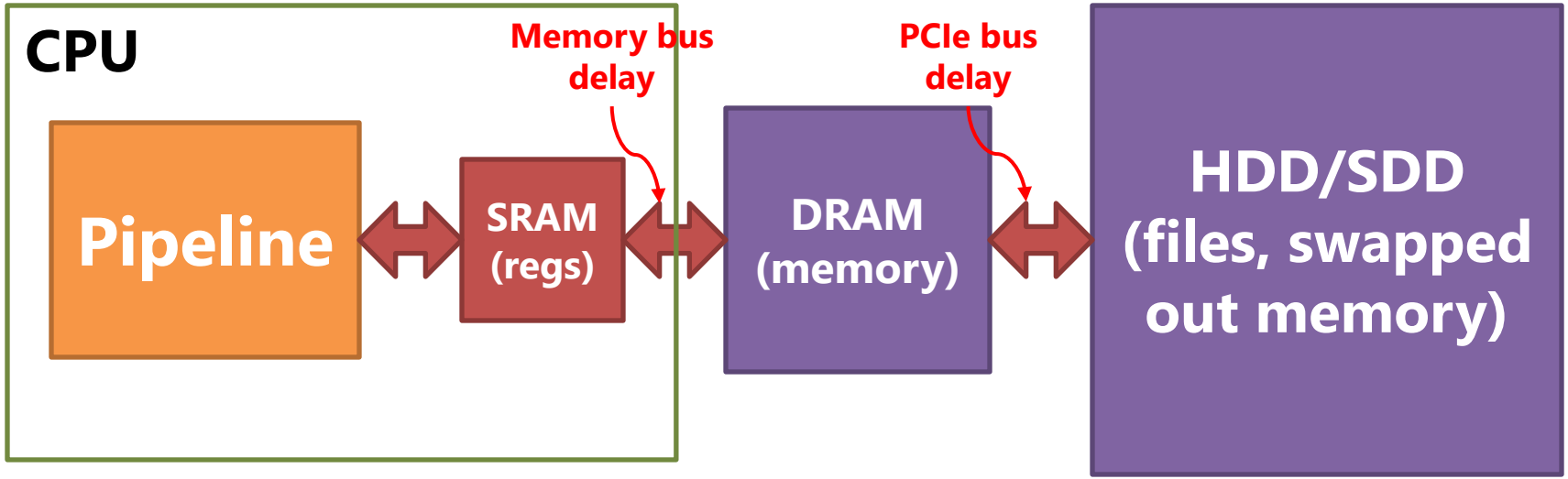


- Access Time = Memory Cell Delay + Address Decode Delay + **Wire Delay**
- With high capacity, Wire Delay (word lines + bit lines) starts to dominate
- Wire Delay ∝ Wire Length ∝ Memory Structure Dimensions
- DRAM density > SRAM density → DRAM Wire Delay < SRAM Wire Delay

The Memory Hierarchy

System Memory Hierarchy

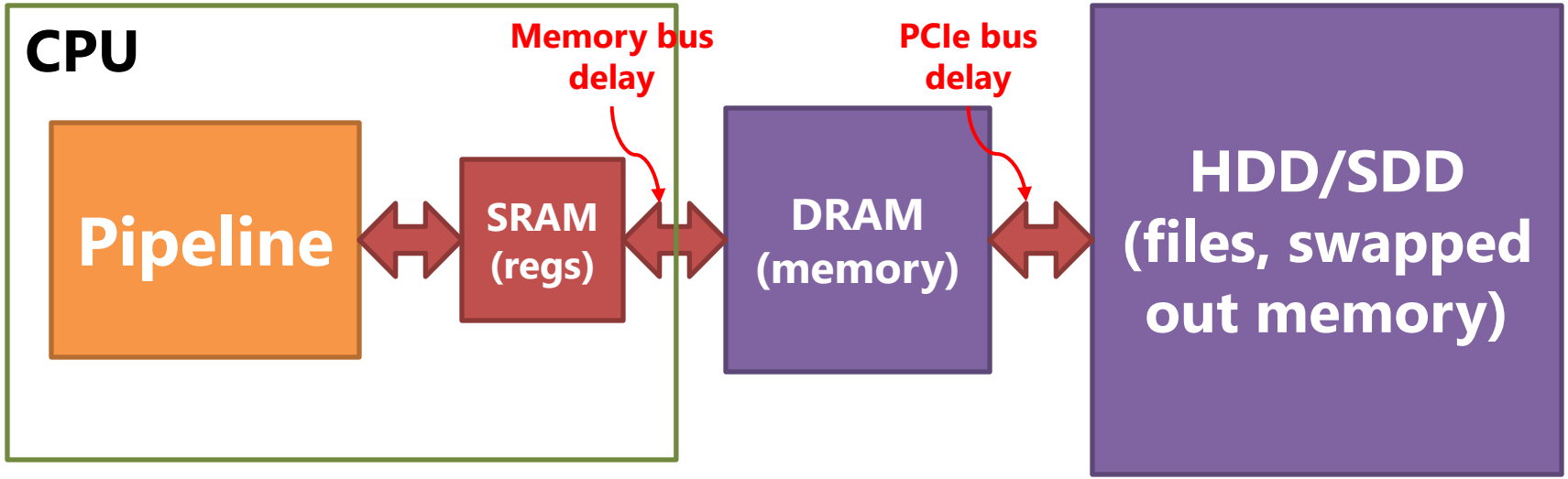
- Use small fast memory to store frequently used data
- Use big slow memory to store infrequently used data



- Registers: used frequently so stored in small SRAM
- Memory pages used frequently: stored in big DRAM
- Memory pages used infrequently: stored in HDD/SDD (in swap space)
- Note: Memories outside CPU chip suffers from bus delay as well

System Memory Hierarchy

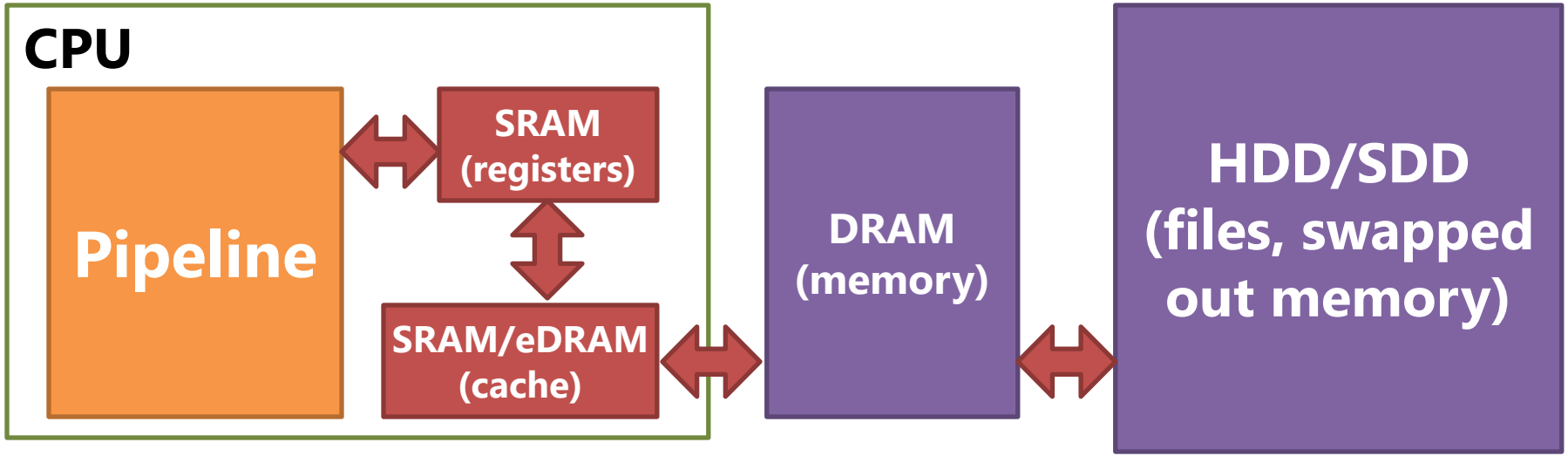
- Use small fast memory to store frequently used data
- Use big slow memory to store infrequently used data



- Memory Wall: memory access is much slower compared to registers
- Q: Can we make memory access speed comparable to register access?

System Memory Hierarchy

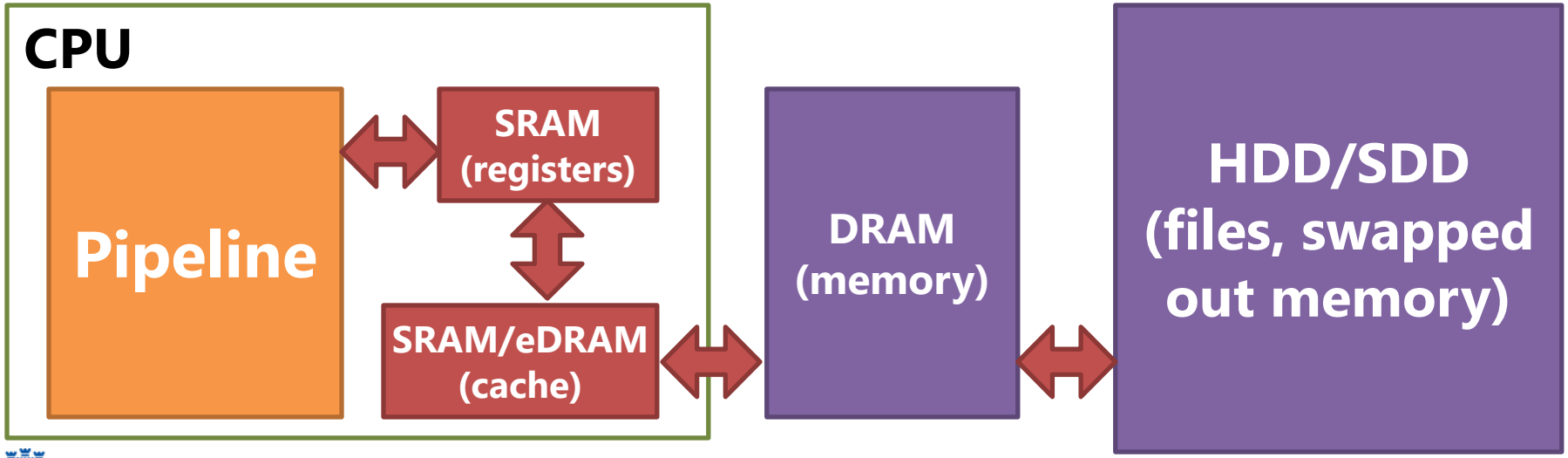
- Use small fast memory to store frequently used data
- Use big slow memory to store infrequently used data



- Memory Wall: Memory access is much slower compared to registers
- Q: Can we make memory access speed comparable to register access?
  - **Cache**: small, fast on-chip memory for frequently used data
  - Caches can be either SRAM or eDRAM depending on its size.

Caching happens at multiple levels of the hierarchy

- **Caching**: keeping a temporary copy of data for faster access
- DRAM is in a sense also caching frequently used pages from swap space
  - We are just extending that idea to bring cache data inside the CPU!
- Now instructions like `lw` or `sw` never directly access DRAM
  - They first search the cache to see if there is a **hit** in the cache
  - Only if they **miss** will they access DRAM to bring data into the cache

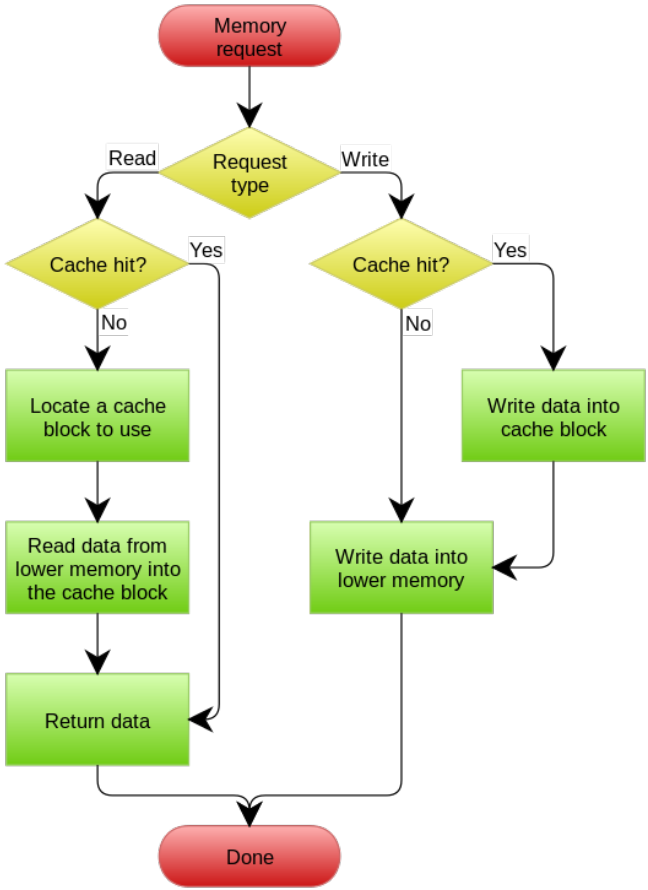


Cache Flow Chart

- **Cache block**: unit of data used to cache data
  - What **page** is to memory paging
  - Cache block size is typically multiple words (e.g. 32 bytes or 64 bytes. You'll see why.)

- **Good**: **Memory Wall** can be **surmounted**
  - On cache hit, no need to go to DRAM.

- **Bad**: MEM stage has **variable latency**
  - Typically, only a few cycles if cache hit
  - More than a 100 cycles if cache miss! (Processor must go all the way to DRAM.)
  - Makes performance very **unpredictable**



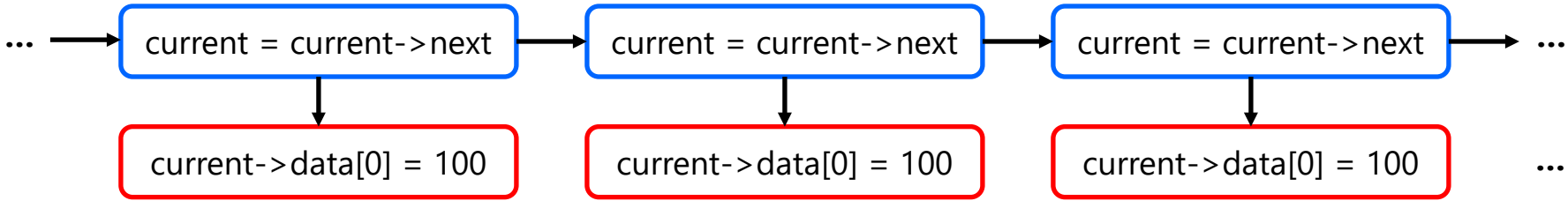
Store vs. Load Memory Accesses

Loads have more impact on performance

- Consider this loop that updates data of each node in a linked list:

```
node_t* current = head;
for(int i=0; i<ACCESSES; i++) {
  if(current == NULL) current = head;          // reached the end
  else {
    current->data[0] = 100;                    // store to node data
    current = current->next;                   // load next node address
}
```

- Which would have more impact on performance? The load or the store?
  - A: The load because it is on the **critical path**.



Loads have more impact on performance

- Loads produce values needed for computation to proceed
  - **Stalled loads delay computation** and possibly the critical path

- Stores write computation results back to memory
  - As long as the results are written back eventually, no big hurry
  - Pending stores are maintained in the **store queue** until they are drained
    - Store cache misses do not stall CPU (unless store queue overflows)

- What if the next computation reads from a pending store?
  - First check the write buffer and read in the value if it's there
  - Again, performing the store is not on the critical path

How Store Queue Drains Stores to Memory

- `sw p0,4(p1)` is about to commit. `4(p1)` == `0xdeadbeef`, `p0` == `10`
- Stores ahead of it have yet to drain into L1 d-cache

L1 d-cache

…
…
…
…

Architectural Register File

t0	s0	a0
t1	s1	a1
t2	s2	a2
…	…	…

Store Queue

	Address	Value	Committed?
Draining	…	…	1
	…	…	1
Unretired	0xdeadbeef	10	0
	…	…	0

Re-Order Buffer (ROB)

instruction	dest	done?
sw p0,4(p1)	-	Y
…		
…		
…		

commit in order

How Store Queue Drains Stores to Memory

- Store retires and `0xdeadbeef` with value `10` is pending in write buffer
- While draining, it encounters a miss in L1 cache but it does not stall pipeline

L1 d-cache

…
… **Miss!**
…
…

Architectural Register File

t0	s0	a0
t1	s1	a1
t2	s2	a2
…	…	…

Store Queue

	Address	Value	Committed?
	…	…	1
Draining	…	…	1
	0xdeadbeef	10	1
Unretired	…	…	0

Re-Order Buffer (ROB)

instruction	dest	done?
…		
…		
…		
…		

commit in order

How Store Queue Drains Stores to Memory

- All loads check Store Queue first before accessing L1 d-cache
- Until **0xdeadbeef** drains, it provides value in place of memory

L1 d-cache
…
… **Miss!**
…
…

Architectural Register File		
t0	s0	a0
t1	s1	a1
t2	s2	a2
…	…	…

Store Queue		
Address	Value	Committed?
…	…	1
…	…	1
0xdeadbeef	10	1
…	…	0

Draining

Unretired

Re-Order Buffer (ROB)		
instruction	dest	done?
…		
…		
…		
…		

commit in order

So are stores never on the critical path?

- In real life, store queue is limited and can suffer structural hazards.
  - But optimal store queue size can be estimated using Little's Law.

- **Little's Law**: $L = \lambda \times W$
  - L = Line length (number of store queue entries for draining)
  - λ= Arrival rate (average store frequency in instruction stream)
  - W = Wait time (average time store spends draining in store queue)
- For M/M/1 queue, $W = S / (1 - \rho)$
  - S = Service time (average time store takes to drain into cache write port)
  - $\rho = \lambda \times S$ = Utilization (average utilization of cache write port)
- Then, $L = \lambda \times W = \lambda \times S / (1 - \rho) = \rho / (1 - \rho)$
  - Caveat: Only holds if memory bandwidth is sufficient ($\rho < 1$)

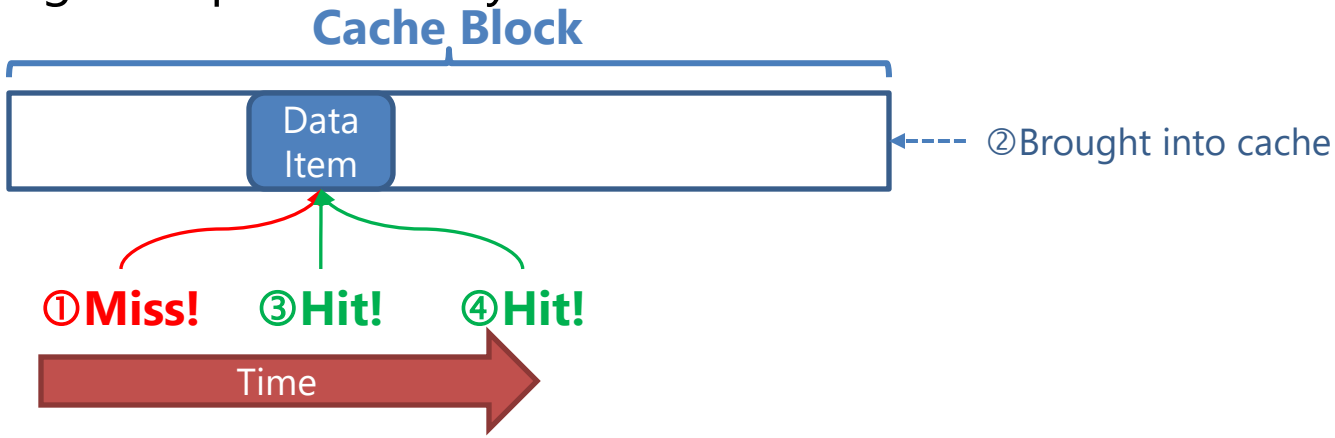
- If 1 store every 4 cycles and store service time is 3 cycles:
  - $\rho = 1/4 * 3 = 0.75$, $L = \rho / (1 - \rho) = 0.75 / (1 – 0.75) = 3$ entries for draining

- Hence, loads are more important in analyzing performance

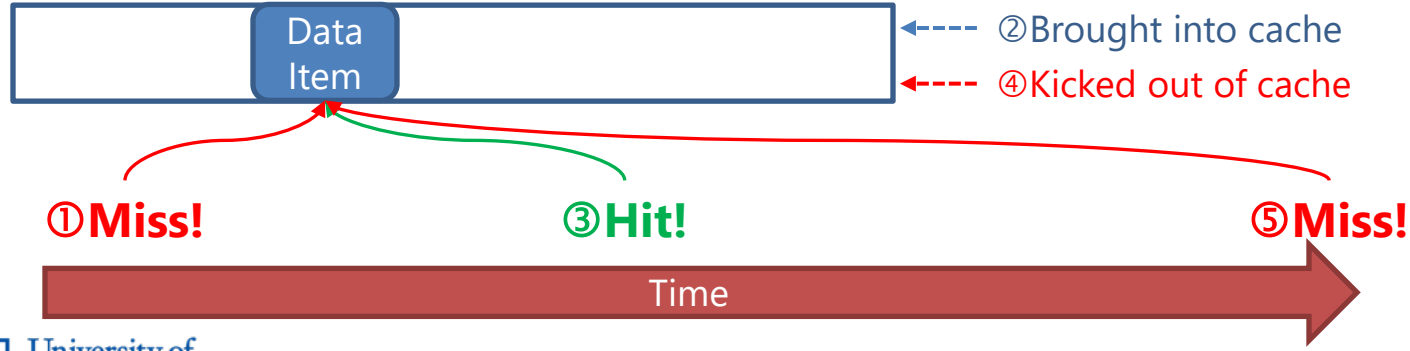
Temporal Locality and Spatial Locality

Cache Locality: Temporal Locality

- **Temporal Locality**: How close together are accesses in **time**?
- High temporal locality:



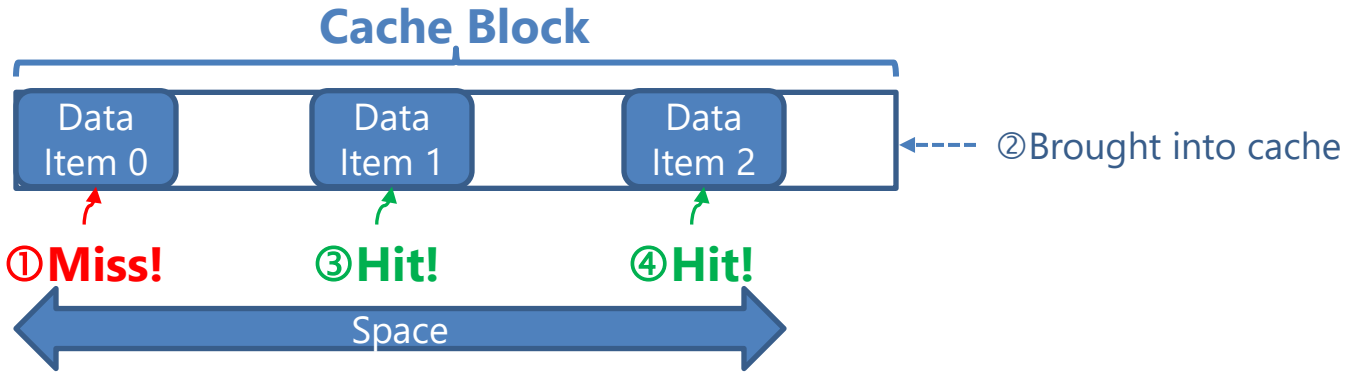
- Low temporal locality:



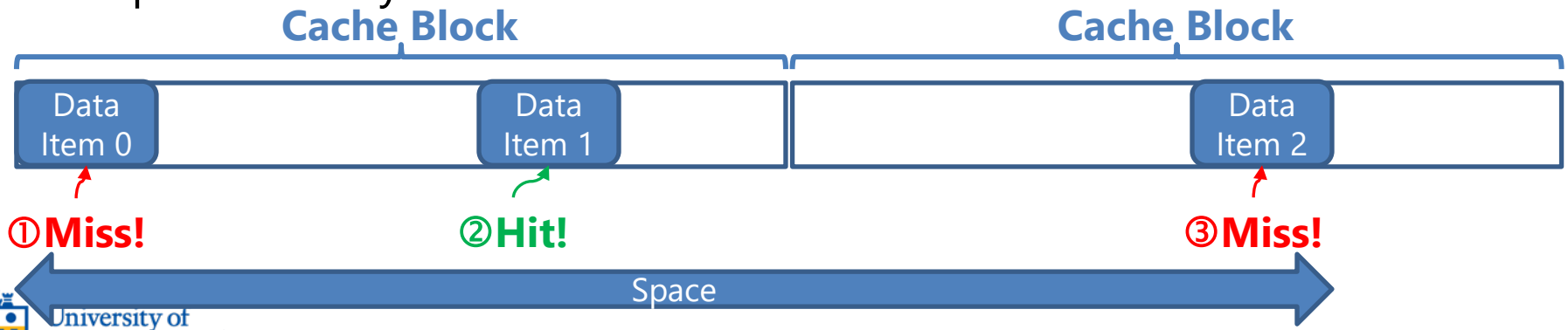
Cache Locality: Spatial Locality

- **Spatial Locality**: How close together are accesses in **space**?

- High spatial locality:



- Low spatial locality:



Cache Locality: Temporal and Spatial

- Caching works only if there is **locality** in program data accesses
  - **Temporal locality**
    - How close together in **time** are accesses to the **same item**
    - 1st access will miss but following accesses can hit in the cache
    - E.g., variables on the stack are accessed close together in time
  - **Spatial locality**
    - How close together in **space** are accesses to **different items**
    - 1st access will miss but bring in an entire cache block
    - Accesses to other items within same cache block will hit
    - E.g., object fields or array elements often accessed consecutively

- Locality, like ILP, is a **property of the program**

Cold Misses and Capacity Misses

- **Cold miss** (a.k.a. **compulsory miss**)
  - Miss suffered when data is accessed for the **first time** by program
  - Cold miss since cache **hasn't been warmed up** with accesses
  - Compulsory miss since there is no way you can hit on the first access

- **Capacity miss**
  - Miss suffered when data is accessed for the **second or third times**
  - This miss occurs because **data was replaced** to make space
  - Not compulsory since with more capacity, miss wouldn't have happened
  - Capacity determines how much **locality** you can leverage

Reducing Cold Misses

- How do you reduce cold misses? Is it even possible?
  - Yes! By taking advantage of **spatial locality**.
  - Have a larger cache block so you bring in other data items on a miss.
  - If those other items are accessed, they will hit, even on first access!

- **Large cache blocks:**
  + Potentially **reduce cold misses** (by leveraging spatial locality)
  + Potentially **reduce capacity misses** (again, leveraging spatial locality)
  – Potentially **increase capacity misses** (pollutes cache with useless items)

- So, larger or smaller cache blocks?
  - It all depends on how much spatial locality each program has.

Reducing Capacity Misses

- How do you reduce capacity misses?
  - Easy! Increase capacity to take advantage of **temporal locality**.

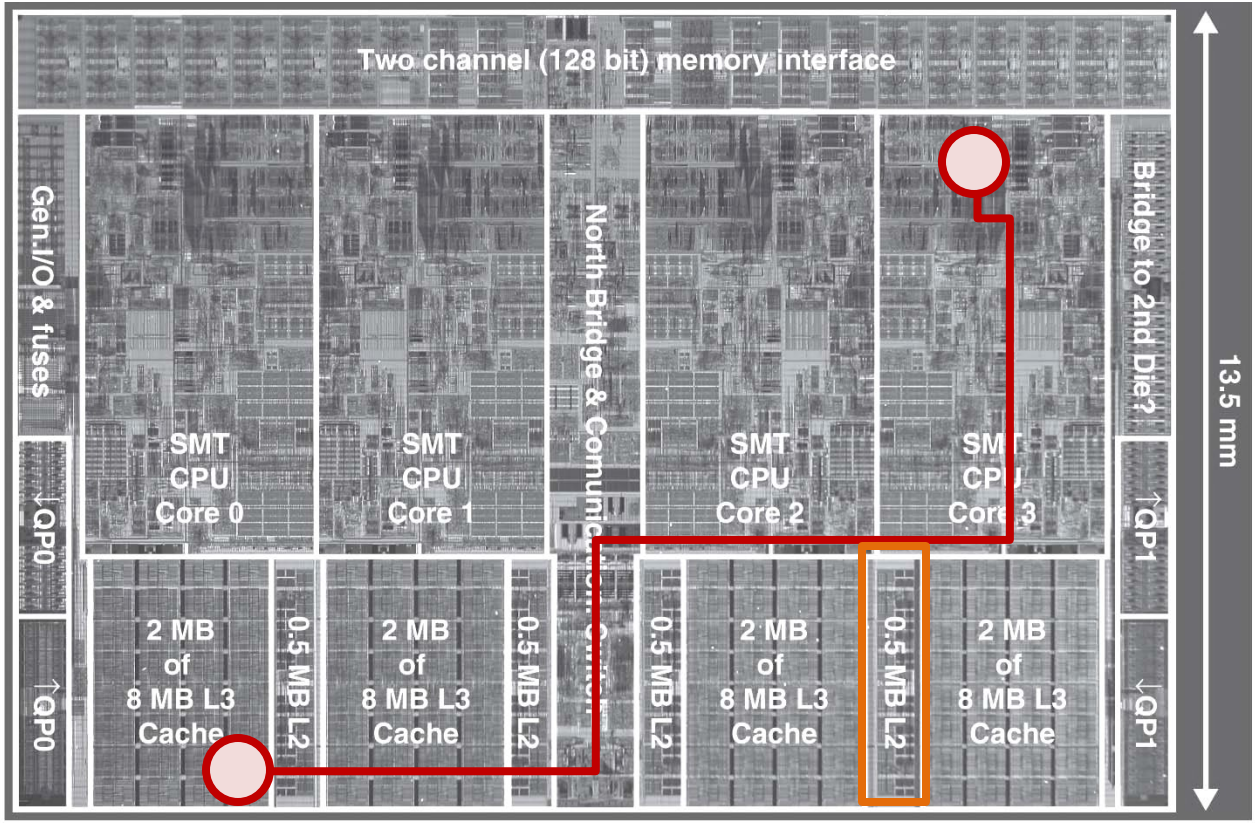
- **Large caches**:
  + Potentially **reduce capacity misses** (by leveraging temporal locality)
  – Potentially **increase cache access delay** (due to longer wires)

- So, larger or smaller caches?
  - Again, it depends on how much temporal locality each program has.

Bigger caches are slower

- Below is a diagram of an Intel Nehalem CPU

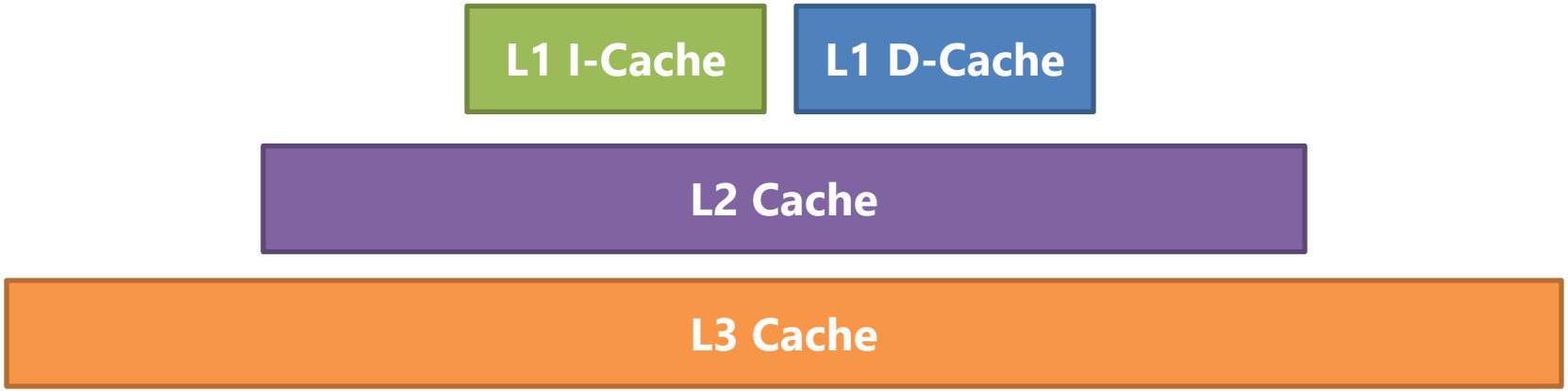
- How long do you think it takes for data to make it from here…
- …to here?
- It must be routed through all this.
- Can we cache the data in the far away "L3 Cache" to a nearby "L2 Cache"?



Multi-level Caches

Multi-level Caching gives choice of where to put data

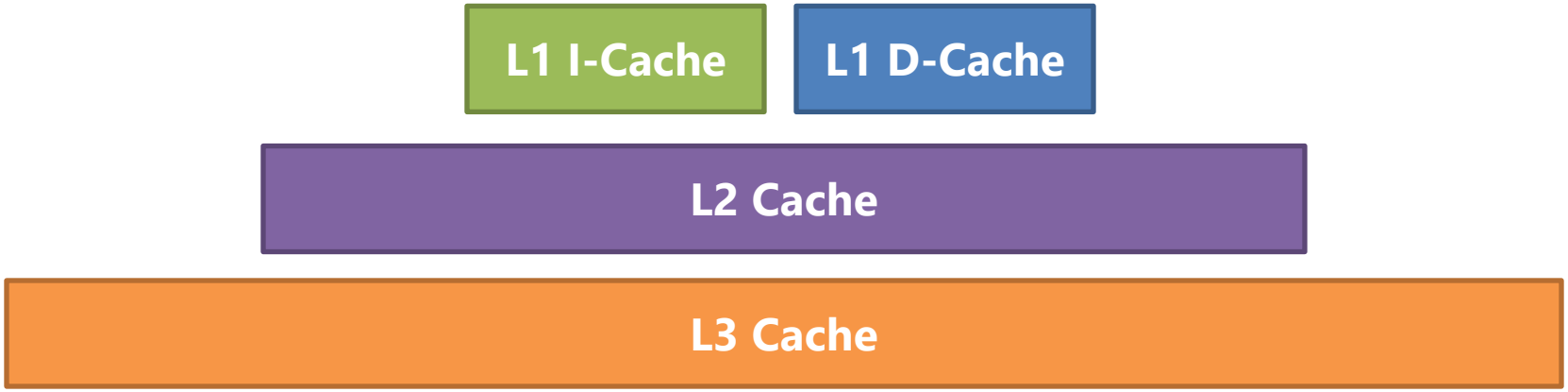
- Put data with **high temporal locality** in **small** caches.
- Put data with **low temporal locality** in **bigger** caches.



- At above is the structure of the thompson.cs.pitt.edu Xeon CPU.
- L1 cache: **Small** but **fast**. Interfaces with CPU pipeline MEM stage.
  - Split to i-cache and d-cache to avoid structural hazard.
- L2 cache: Middle-sized and middle-fast. Intermediate level.
- L3 cache: **Big** but **slow**. Last line of defense against DRAM access.

Typical use of multi-level caches in software

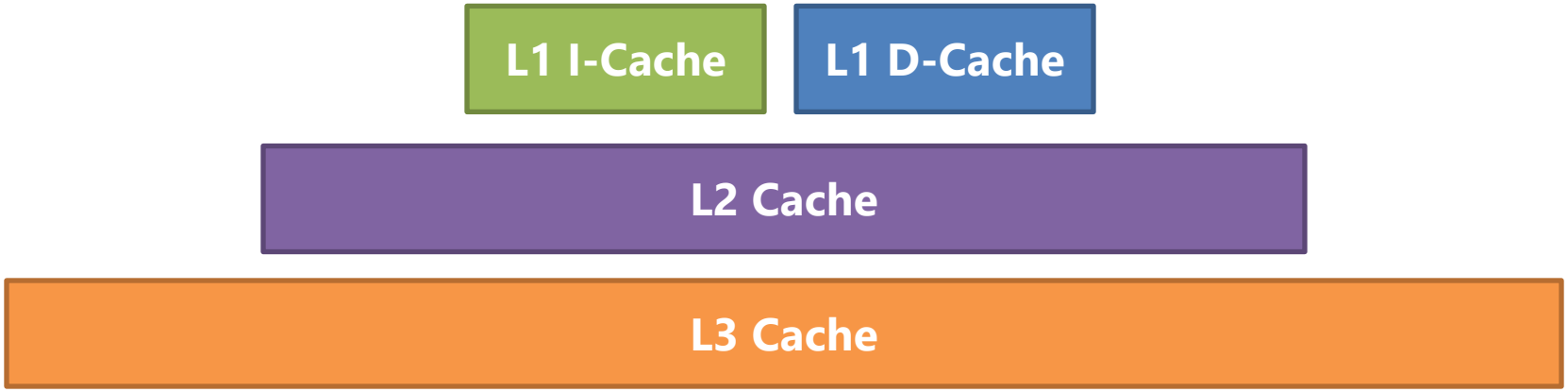
- Goal: place data at optimal cache level depending on temporal locality.



- L1 cache: **Small** data set that is accessed **frequently**
  - Local variables in stack memory, code within current loop
- L2 cache: **Medium** data set that is accessed **less frequently**
  - Currently accessed smaller data structures, code within current function
- L3 cache: **Big** data set that is accessed **even less frequently**.
  - Currently accessed bigger data structures, code within current module
- Up to HW cache replacement algorithm to place data sets appropriately

Multi-level caches may use hybrid memories

- Smaller caches use SRAM / bigger caches use DRAM.



- L1 cache, L2 cache: typically uses SRAM
  - Small enough that memory cell delay is important
- L3 cache: May use eDRAM (embedded DRAM) over SRAM
  - Can fit in much more eDRAM than SRAM for same silicon area
  - At high capacity, memory cell delay is less significant

Memory access latencies displays step function

- Figure from NASA report reverse engineering Intel CPU cache latencies
  - Ref: https://www.nas.nasa.gov/assets/pdf/papers/NAS_Technical_Report_NAS-2015-05.pdf
- Plots average load latency while increasing memory working set size

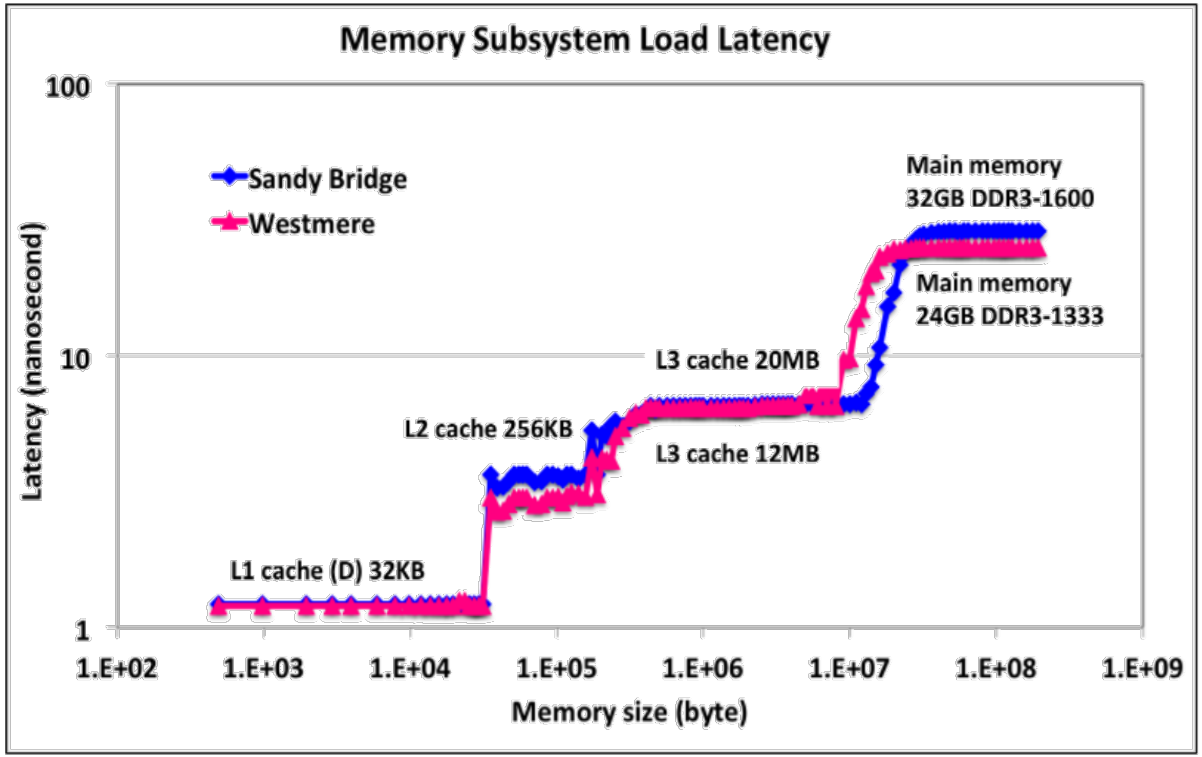
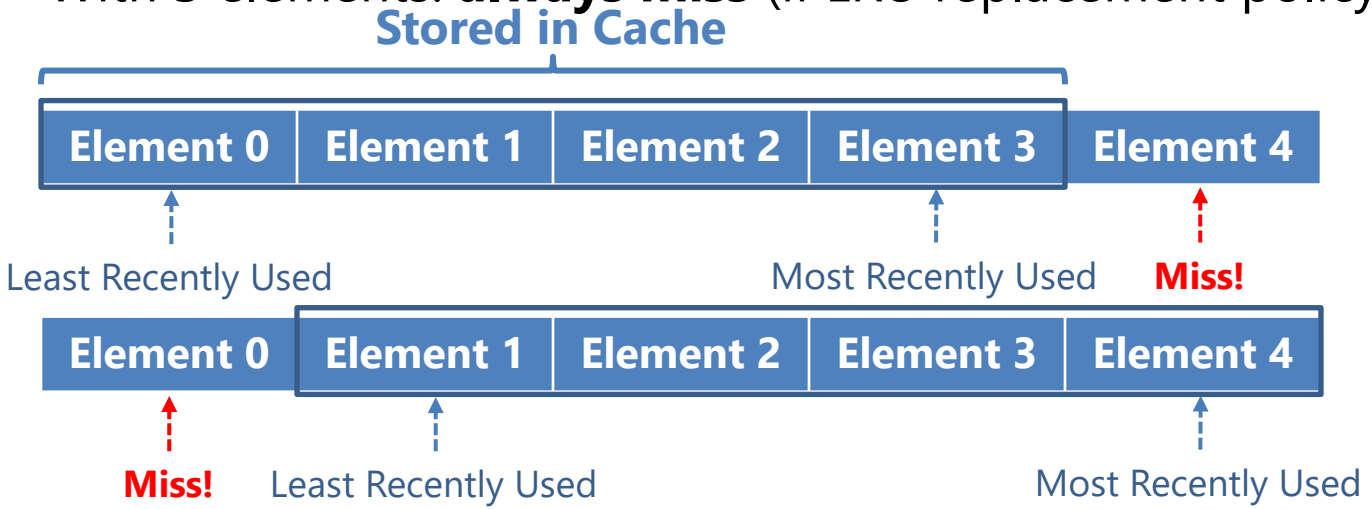


Figure 2. Memory latency of Westmere and Sandy Bridge.

Why the Step Function?

- **Working set**: amount of memory program accesses during a phase
  - Typically dominated by data structures that program accesses
- When working set **overflows** cache capacity, start to see **cache misses**
  - Miss increase can be drastic, almost like a **step function**
  - Suppose cache can hold 4 array items, over which we iterate repeatedly
    - With **4** elements: **always hit** (since entire array is contained in cache)
    - With **5** elements: **always miss** (if LRU replacement policy is used)